

Deep Learning: Practical Work 2 - Handling Open-Set-Recognition

Clàudia Boixader & Óscar Cubeles

January, 2026

1 Description

The objective of this practical work is to study the behavior of deep learning image recognition models in an open-set recognition scenario, where models must handle both seen (in-distribution) and unseen (out-of-distribution) classes. Traditional image classifiers are trained under a closed-set assumption, meaning that all classes encountered during inference are known in advance. However, in real-world scenarios, models may encounter objects that were not present during training, leading to unreliable or overconfident predictions.

In particular, this work investigates traffic sign recognition in closed-set and open-set scenarios using deep learning models. In the first part, several object recognition architectures are evaluated, including an implemented model VGG16 from the Keras, a custom convolutional neural network, and a custom residual network designed from scratch. These models are trained and validated on seen classes to assess classification performance and confidence behavior, with particular attention to class-wise performance differences and prediction uncertainty. Additionally, the models are evaluated on unseen classes to analyze their prediction behavior outside the known data distribution, focusing on the assigned class labels and the confidence levels of these predictions.

In the second part of the project, the focus shifts to open-set recognition: an out-of-distribution detection approach is implemented using several autoencoders trained exclusively on in-distribution data, and its effectiveness is evaluated on a test set containing both seen and unseen instances, ensuring that no training samples are reused during evaluation.

1.1 Datasets

The primary dataset used for training and validation is the German Traffic Sign Recognition Benchmark (GTSRB). This dataset contains images of German traffic signs grouped into 43 different classes. See Figure 1 for the whole diversity of classes and Figure 2 to see some specific image examples.

Each class represents one traffic signs and within class images exhibit significant variability in terms of illumination, resolution, viewpoint, and background, making the dataset suitable for evaluating real-world robustness. The original dataset includes only train and test folders. Having 39209 instances on the train set and 12631 on the test set. From these two folders, we first split the original train folder into train and validation set in an 80/20 split. Finally, a subset of 1000 images of the test dataset are reserved for evaluation.

In order to simulate an open-set scenario, we also incorporated an additional dataset from Kaggle - Traffic Sign Dataset Classification - from which we selected samples that were not included in the GTSRB main dataset (for more information about the dataset, see [this link](#)).



Figure 1: German traffic sign recognition benchmark

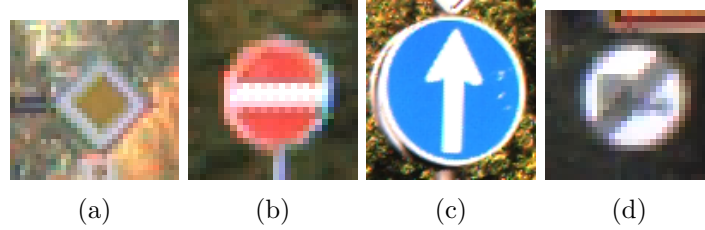


Figure 2: GTSRB seen samples examples

These signs differ in appearance, design, or country of origin from those in the GTSRB dataset. See Figure 3 to see some examples of images from the dataset.

The classes in this dataset are only used for evaluation. In the first part of the work, a small subset of this dataset, containing only 32 samples is used to assess the VGG16, ResNet, and CNN models performance on unseen samples. For the second part, it is combined with the GTSRB test set to create a new test dataset containing both seen and unseen samples, which is then used for evaluating the OOD detection model. Specifically, this new dataset contains 1000 seen samples extracted from the test GTSRB dataset and xxx unseen samples from the Traffic Sign Dataset Classification dataset.

2 Architectures for in Distribution Detection

To evaluate the impact of architectural design on performance and generalization, three different convolutional neural network architectures were implemented and compared. These include a standard pre-implemented model and two custom-designed networks.

2.1 VGG16

The first architecture used in this work is VGG16, implemented using the Keras deep learning framework. VGG16 is a well-established convolutional neural network characterized by its deep yet simple architecture, consisting of sequential convolutional blocks with small 3×3 kernels.

The network is composed of five convolutional stages, each containing multiple convolutional layers followed by ReLU activations and max-pooling operations. As the depth increases, the number of filters grows progressively, enabling the extraction of increasingly abstract visual features. After the convolutional backbone, fully connected layers are used to perform classification.



Figure 3: GTSRB seen samples examples

For this project, the VGG16 architecture serves as a strong baseline, leveraging its proven feature extraction capabilities. The model is adapted to the traffic sign classification task by replacing the original classification head with a task specific output layer corresponding to the number of seen classes.

2.2 Basic CNN

In addition to the standard VGG16 architecture, a custom CNN was designed and trained from scratch. This model was developed to provide a lightweight and more interpretable alternative, tailored specifically to the characteristics of the traffic sign datasets.

The network consists of a sequence of convolutional layers with increasing numbers of filters, each followed by batch normalization and ReLU activation to stabilize training and improve convergence. Max-pooling layers are used to progressively reduce spatial resolution, while preserving discriminative features. Dropout regularization is incorporated to mitigate overfitting, particularly given the limited variability of some traffic sign classes.

After the convolutional feature extraction stage, the model employs fully connected layers to map the learned representations to class scores. Compared to VGG16, this architecture has fewer parameters and a shallower depth, making it computationally efficient while still achieving competitive performance.

2.3 ResNet

The third architecture implemented in this work is a custom residual network, inspired by the principles of ResNet architectures. Unlike standard feedforward CNNs, this model incorporates residual connections, which allow the input of a layer to bypass one or more convolutional transformations.

The network is composed of multiple residual blocks, each containing two convolutional layers with batch normalization and ReLU activation. A shortcut connection adds the block input directly to its output, enabling more effective gradient flow during training. When changes in spatial resolution or feature dimensionality occur, a 1×1 convolution is applied to the shortcut path to ensure dimensional compatibility.

As the network depth increases, the number of filters is progressively expanded, allowing the model to learn richer feature representations. At the end of the network, global average pooling aggregates spatial information, followed by a fully connected output layer producing class logits for the seen categories.

This residual formulation helps mitigate the vanishing gradient problem and supports more stable training compared to the custom CNN, making it a suitable candidate for improved generalization.

3 Architectures for OOD Detection

The primary objective is to detect out-of-distribution (OOD) traffic signs using autoencoder-based approaches. Autoencoders are trained exclusively on the training split of the Ger-

man Traffic Sign Recognition Benchmark (GTSRB) dataset to learn the distribution of in-distribution traffic signs. The training set comprises 31367 samples, 7842 of which are allocated for validation. The idea behind this approach is that by learning to reconstruct only the in-distribution samples during training, these autoencoders can subsequently identify OOD traffic signs by exhibiting higher reconstruction errors when presented with unseen classes. For this task three different autoencoders have been developed.

3.1 Convolutional Autoencoders

For this task, two Convolutional Autoencoders with two levels of complexity have been developed to evaluate the trade-off between model simplicity and representational capacity for OOD detection. The motivation is to establish a simple baseline architecture and then progressively add complexity to investigate whether regularization and deeper feature processing improve OOD detection performance.

3.1.1 Simple Convolutional Autoencoder

The Simple Convolutional Autoencoder employs a straightforward encoder-decoder architecture without regularization techniques such as batch normalization or dropout. The encoder consists of four sequential convolutional layers with stride-2 downsampling and ReLU activations, progressively reducing spatial dimensions from 64×64 to 32×32 , 16×16 , 8×8 , and finally 4×4 pixels. Simultaneously, the feature channels expand from 3 to 32, 64, 128, and 256 channels respectively. The bottleneck layer compresses the representation to 64 channels using a single 1×1 convolutional operation. The decoder mirrors the encoder structure exactly, employing four transposed convolutions with stride-2 upsampling to progressively restore the original spatial dimensions. ReLU activations are applied throughout the decoder, with a final sigmoid activation applied to the output layer to constrain reconstructed pixel values to the range $[0, 1]$. The idea of this architecture is to have a simple autoencoder to test if a simple approach was able to represent in-distribution traffic signs. The lack of regularization allows the network to learn a tighter mapping to the training distribution, potentially resulting in higher reconstruction errors for OOD samples that deviate from the learned patterns. However, this unregularized approach risks overfitting to training-specific characteristics, which may not generalize well if OOD samples share some visual similarities with the training data.

3.1.2 Deep Convolutional Autoencoder

The Deep Convolutional Autoencoder extends this baseline approach by incorporating batch normalization, dropout regularization to improve robustness and generalization. Instead of single-layer convolutions, each encoder block contains two sequential convolution operations interleaved with batch normalization and ReLU activations, allowing for more refined feature extraction at each spatial resolution. The spatial downsampling progression remains identical ($64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4$), but each block incorporates dropout layers with rates of 0.2 to 0.3 to prevent overfitting and encourage feature robustness. The bottleneck represents a significant departure from the Simple Convolutional Autoencoder, rather than compressing to 64 channels, the Deep variant reduces to only 16 channels through a 1×1 convolution, enforcing a much tighter information bottleneck. This forces the network to learn only the most discriminative features of the training distribution. The decoder mirrors the encoder structure using the same block-based design with transpose convolutions, batch normalization, and dropout applied throughout for consistency. The final sigmoid activation constrains output values to $[0, 1]$. By combining regularization techniques with a tighter bottleneck, the Deep Convolutional Autoencoder aims to achieve better generalization to unseen classes while maintaining the ability to detect OOD samples through elevated reconstruction errors. The

hypothesis is that the regularization prevents overfitting to irrelevant training details, allowing the network to focus on essential traffic sign features that generalize better to OOD detection tasks.

3.2 Simple Sparse Autoencoder

The Simple Sparse Autoencoder extends the Convolutional Autoencoders by incorporating sparsity constraints through L1 activity regularization, similar to how the Deep variant adds regularization techniques. Like the Deep Convolutional Autoencoder, it maintains the identical spatial downsampling progression ($64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4$) but employs a different regularization strategy. Instead of using batch normalization and dropout, the Simple Sparse Autoencoder applies L1 activity regularization at the bottleneck layer with a weight of 10^{-5} . This regularization constraint encourages sparse activation patterns, where only a small subset of neurons are active at each spatial location. The encoder consists of four sequential convolutional layers with stride-2 downsampling, progressively increasing feature channels from 3 to 32, 64, 128, and 256 channels, which mirrors the channel progression of the Simple Convolutional Autoencoder. The bottleneck compresses the representation to 64 channels, maintaining the same compression level as the original baseline. The decoder employs four transposed convolution layers to upsample back to the original input dimensions, with sigmoid activation applied to the final output. The idea is that by incorporating sparsity regularization, the Simple Sparse Autoencoder aims to learn compact, interpretable feature representations similar to the regularization goals of the Deep variant. So, sparse activations should force the network to discover and utilize only the most essential features of in-distribution traffic signs, potentially improving its ability to identify OOD samples through elevated reconstruction errors when faced with unfamiliar visual patterns.

4 Result Analysis

4.1 Part 1: Object Recognition Models

4.1.1 Training Setup

This section presents the performance of the three developed model for object recognition - VGG16, CNN and ResNet - trained on the GTSRB dataset. All models were trained on the original training dataset of 39,209 samples, split into 80% for training (31,367 samples) and 20% for validation (7,842 samples). Moreover, their performance was finally evaluated on unseen samples using the same 32 samples extracted from the Traffic Sign Dataset Classification Kaggle dataset.

The three experiments were conducted using the hyperparameter configurations summarized in table 1. Three architectures - VGG, a custom CNN, and ResNet - were trained with a fixed input resolution of 64×64 pixels and a batch size of 32. All models were compiled using the Adam optimizer and trained with the Sparse Categorical Crossentropy loss function. The VGG model was trained for 200 epochs, while the CNN and ResNet models were trained for 150 and 100 epochs, respectively, all using a learning rate of $1e^{-4}$.

4.1.2 Evaluation

Before analyzing the unseen data, it is important to first examine the training metrics. Looking at Figure 4, we observe that VGG16 demonstrates the most stable and rapid convergence, reaching nearly 100% accuracy within the first 10 epochs with minimal noise in the loss curve. This indicates strong optimization stability and effective feature learning early in training.

Configuration	VGG	CNN	ResNet
Input Size	(64, 64)	(64, 64)	(64, 64)
Batch Size	32	32	32
Learning Rate	1×10^{-4}	1×10^{-4}	1×10^{-4}
Number of Epochs	200	150	100
Optimizer	Adam	Adam	Adam
Loss Function	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy
Open-set Examples per Class	10	10	10

Table 1: Hyperparameter configurations for the object recognition models

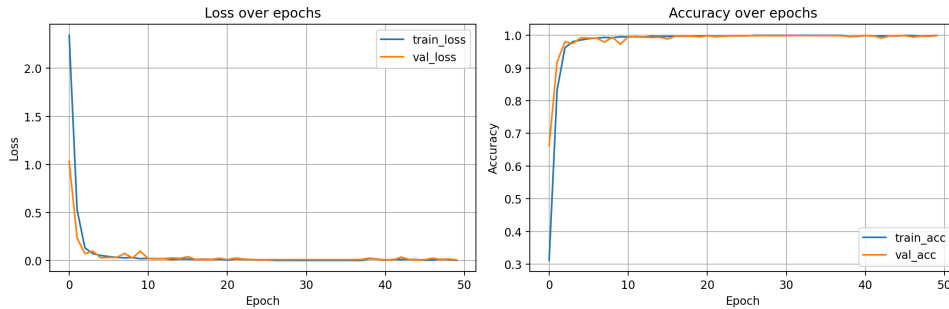


Figure 4: Accuracy and Loss metrics: VGG model

Figure 5 illustrates the ResNet training dynamics, where high overall performance is achieved; however, the validation loss curve shows noticeable spikes around epochs 72 and 90. These spikes suggest periodic instabilities or sensitivity to specific validation batches, although the model consistently recovers shortly afterward.

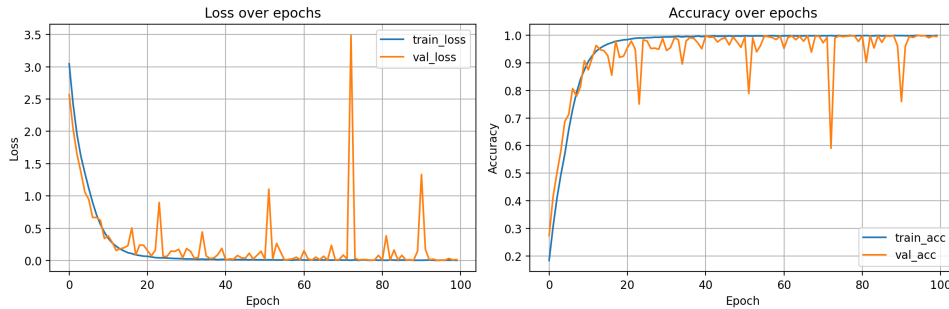


Figure 5: Accuracy and Loss metrics: ResNet model

In contrast, Figure 6 shows that the Simple CNN converges much more slowly, requiring nearly 150 epochs to reach its performance plateau at approximately 96–97% accuracy, indicating a lower learning capacity and less efficient feature extraction.

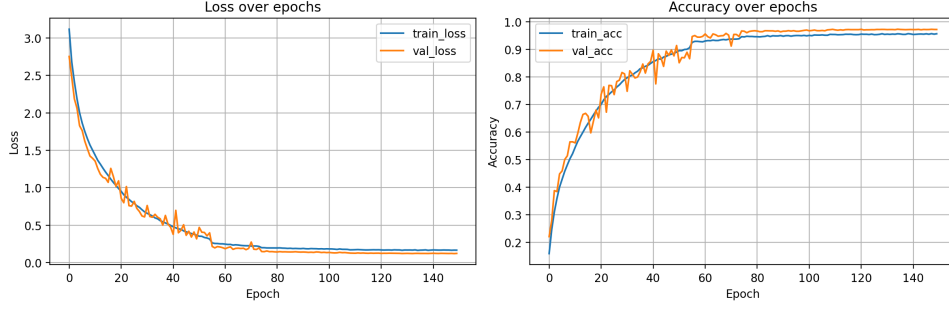


Figure 6: Accuracy and Loss metrics: CNN model

Further details on the performance for seen classes can be found in the appendix, specifically in Figures 10, 12, and 11, which present the per-class results for the VGG16, CNN, and ResNet models, respectively.

One of the objectives of this study is to evaluate how these models behave when exposed to out-of-distribution (OOD) samples - data they were not trained on. Ideally, a robust model should express uncertainty in such cases, reflected by low confidence scores and high entropy values. Tables 2, 4 and 6 summarize the mean confidence, mean entropy, and proportion of high-confidence predictions (≥ 0.9) for unseen samples across all three models. Despite differences in architecture, all models exhibit a pronounced overconfidence problem: approximately 44–47% of unseen samples are predicted with very high confidence. This behavior highlights a well-known limitation of softmax-based classifiers, which tend to amplify the most dominant logit even when the input does not belong to any known class.

Among the three models, VGG16 shows relatively better OOD sensitivity. As reported in Table 2, it achieves the lowest mean confidence (0.7968) and the highest mean entropy (0.7906) on unseen data, suggesting that it is marginally more uncertain than both ResNet and the custom CNN. Although the improvement is modest, this behavior indicates that VGG16 provides slightly stronger warning signals when encountering unfamiliar inputs, making it the most suitable candidate for OOD detection in this experimental setup.

Metric	Confidence	Entropy
Mean	0.7968	0.7906
Std	0.2522	0.9830
Min	0.1427	0.0000
Max	1.0000	3.3007

Additional Measures	Value
Total unseen samples	32
Conf. Entropy Corr.	-0.9827
High conf. (> 0.9)	46.88%

Table 2: Unseen Classes: Confidence and Entropy Statistics - VGG16 Model

Rank	Class ID	Percentage (%)
1	5	12.50
2	11	12.50
3	2	9.38
4	13	9.38
5	32	9.38
6	20	6.25
7	30	6.25
8	33	6.25
9	38	6.25
10	6	3.12

Table 3: Top Predicted Known Classes for Unseen Samples - VGG16 Model

Metric	Confidence	Entropy
Mean	0.8149	0.5524
Std	0.1834	0.4741
Min	0.3619	0.0026
Max	0.9998	1.8554
Additional Measures		Value
Total unseen samples		32
Conf. - Entropy Corr.		-0.9014
High conf. (> 0.9)		46.88%

Table 4: Unseen Classes: Confidence and Entropy Statistics - CNN Model

Rank	Class ID	Percentage (%)
1	13	18.75
2	1	9.38
3	25	9.38
4	39	9.38
5	2	6.25
6	4	6.25
7	9	6.25
8	12	6.25
9	40	6.25
10	0	3.12

Table 5: Top Predicted Known Classes for Unseen Samples - CNN Model

Metric	Confidence	Entropy
Mean	0.8410	0.4905
Std	0.1566	0.4461
Min	0.5160	0.0000
Max	1.0000	1.4665
Additional Measures		Value
Total unseen samples		32
Conf. - Entropy Corr.		-0.9643
High conf. (> 0.9)		43.75%

Table 6: Unseen Classes: Confidence and Entropy Statistics - ResNet Model

Rank	Class ID	Percentage (%)
1	12	25.00
2	13	15.62
3	2	12.50
4	1	6.25
5	26	6.25
6	34	6.25
7	3	3.12
8	10	3.12
9	11	3.12
10	23	3.12

Table 7: Top Predicted Known Classes for Unseen Samples - ResNet Model

Further insight is provided by the reliability analysis. Figures 7, 8 and 8 present scatter plots of accuracy versus confidence and entropy for unseen classes. Across all models, there is a strong negative correlation between confidence and entropy, confirming the internal consistency of these uncertainty metrics. Notably, the ResNet and VGG16 plots show dense clusters in the top-right region, corresponding to high accuracy and high confidence. This indicates that when unseen samples share visual features with known classes, the models can classify them accurately while still failing to recognize them as novel. A consistent outlier pattern is also visible: Class 0 appears as a low-accuracy, high-entropy cluster in both the ResNet and VGG16 plots, suggesting that this class is either particularly distinct or easily confused with OOD samples at the feature level.

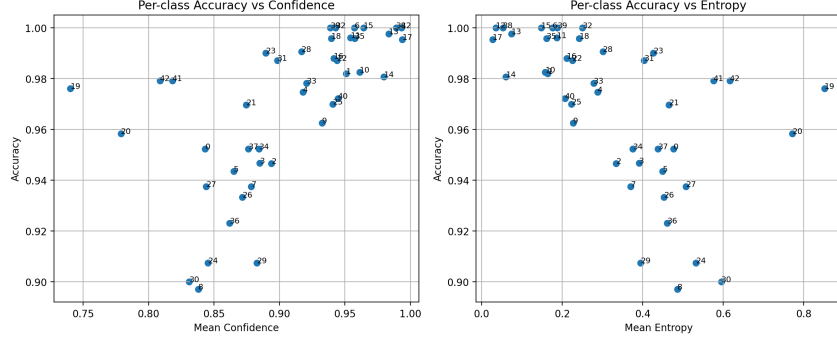


Figure 7: Accuracy versus confidence and entropy for unseen classes: CNN model

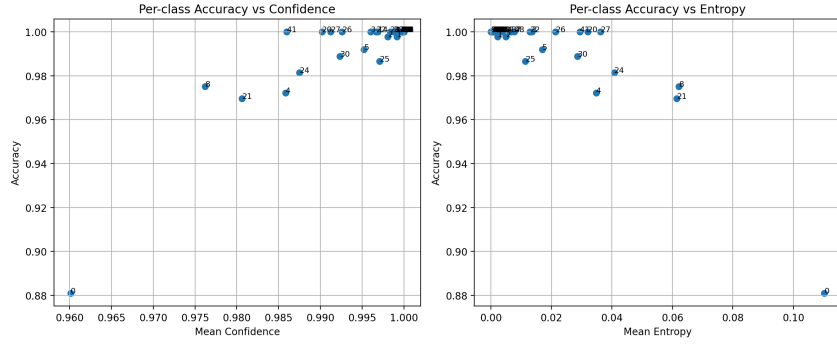


Figure 8: Accuracy versus confidence and entropy for unseen classes: ResNet model

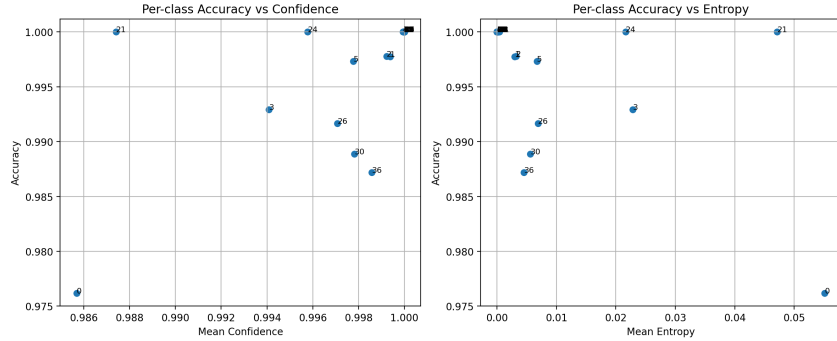


Figure 9: Accuracy versus confidence and entropy for unseen classes: VGG16 model

Misclassification trends further support this interpretation. Tables 5, 7 and 3, which list the top predicted known classes for unseen samples, reveal that the CNN and ResNet most frequently misclassify OOD inputs as Classes 12 and 13, while VGG16 is more frequently attracted to Classes 5 and 11. This pattern suggests a significant feature overlap between the unseen images and these specific known classes.

To complement the previous explanation, we examine Figures 11, 12, and 10, which illustrate how each model predicts a label for the same input image. In each figure, the left side shows a representative sample from the ground-truth class to which the image belongs, while the right side displays a representative sample from the class predicted by the model. All three figures correspond to the prediction of the same unseen image, and the results reveal that, for this specific case, each model assigns a different class label.

Despite these discrepancies, some of the predictions can be partially justified based on shared visual features. For instance, both the CNN and the VGG16 models predict classes

characterized by a rounded shape, suggesting that shape-based features play a significant role in their decision-making process. Furthermore, VGG16 appears to focus on color information, correctly identifying blue as the dominant color in the original image, whereas the CNN may prioritize geometric cues, such as the presence of a circular red shape. In contrast, ResNet produces the most divergent prediction among the three models, indicating that it may rely on a different combination of features or higher-level abstractions when classifying unseen data.



Figure 10: Example ResNet model



Figure 11: Example CNN model



Figure 12: Example VGG16 model

Overall, these results highlight important limitations and practical considerations. While all models achieve high accuracy on known data, their tendency toward overconfidence on unseen samples poses a risk in real-world or safety-critical applications. VGG16, despite being a less modern architecture than ResNet, exhibits slightly better uncertainty behavior.

4.2 Part 2: OOD Detection Performance and Training Configuration

4.2.1 Training Setup

This section presents the OOD detection performance of the three developed Autoencoders trained on the GTSRB dataset. All autoencoders were trained on the original training dataset of 39,209 samples, split into 80% for training (31,367 samples) and 20% for validation (7,842 samples).

Configuration	Simple Conv	Deep Conv	Simple Sparse (L1)
Input Size	(64, 64)	(64, 64)	(64, 64)
Batch Size	32	32	32
Learning Rate	0.001	0.0005	0.001
Number of Epochs	50	100	50
Optimizer	Adam	Adam	Adam
Loss Function	MSE	MSE	MSE + L1

Table 8: Training hyperparameters for the three autoencoder architectures.

The three autoencoders were trained using the hyperparameters presented in Table 8. All models were compiled with the Adam optimizer and optimized using Mean Squared Error (MSE) as the loss function. The Simple Convolutional Autoencoder and Simple Sparse Autoencoder were trained for 50 epochs with a learning rate of 0.001, while the Deep Convolutional Autoencoder was trained for 100 epochs with a reduced learning rate of 0.0005 to allow for finer convergence given its more complex architecture with batch normalization and dropout regularization.

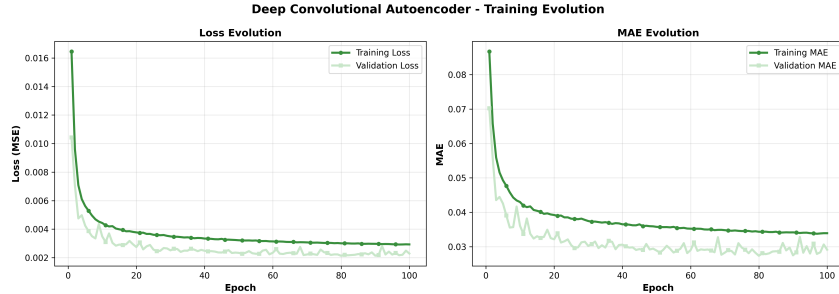


Figure 13: Training evolution for the Deep Convolutional autoencoder

Regarding the training results, both the deep convolutional autoencoder and the simple convolutional autoencoder show a steady and continuous learning behavior throughout training, as reflected by the smooth evolution of the reduced the training loss and Mean Absolute Error (MAE) over epochs. The training dynamics of these models can be seen in Figures 13 and 20, respectively.

On the other hand, the simple sparse autoencoder, shown in Figure 14, displays a rapid reduction of training loss and MAE on the training set. However, the validation metrics present irregular fluctuations and fail to converge. This behavior may suggests that, while the model effectively learns the training data, it may struggle to generalize to unseen samples, potentially leading to unstable performance during inference. This will be analyzed in the next section.

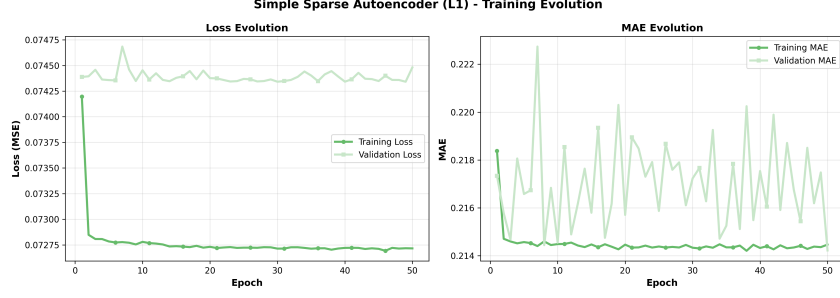


Figure 14: Simple Sparse autoencoder training evolution.

4.2.2 Evaluation Metrics

In order to perform the out-of-distribution (OOD) detection task, it was necessary to define a threshold that separates in-distribution samples from OOD instances. To this end, we adopted a simple statistical criterion based on the reconstruction error observed on the validation set. Specifically, the reconstruction error distribution of the validation data was used to compute a threshold defined as the sum of the mean and one standard deviation of the validation reconstruction error.

$$threshold = \mu_{\text{validation}} + \sigma_{\text{validation}} \quad (1)$$

Consequently, any sample whose reconstruction error exceeds this threshold is considered out-of-distribution in our experimental setup. The resulting threshold was then applied to the test set to distinguish between in-distribution and OOD samples.

Model	Unseen OOD	Seen OOD	Well / Total
Deep Conv. Autoencoder	651/744	203/1055	1503/1799
Simple Conv. Autoencoder	568/744	148/1055	1475/1799
Simple Sparse Autoencoder (L1)	220/744	150/1055	1125/1799

Table 9: Classification and OOD detection performance of the evaluated autoencoder models.

After evaluating the autoencoders on the test set, composed of a total of 1799 instances, of which 1055 belong to seen classes from the GTSRB dataset and 744 correspond to unseen traffic signs, we observe that, at first glance, the deep convolutional autoencoder classifies a larger number of unseen instances as OOD, but also misclassifies a considerable number of seen instances as OOD. In contrast, the simple convolutional autoencoder discards fewer seen instances as false OOD, but classifies more unseen instances as in-distribution. Finally, the simple sparse autoencoder is better at identifying seen instances as in-distribution, although it may lack the ability to effectively reject unseen instances as OOD.

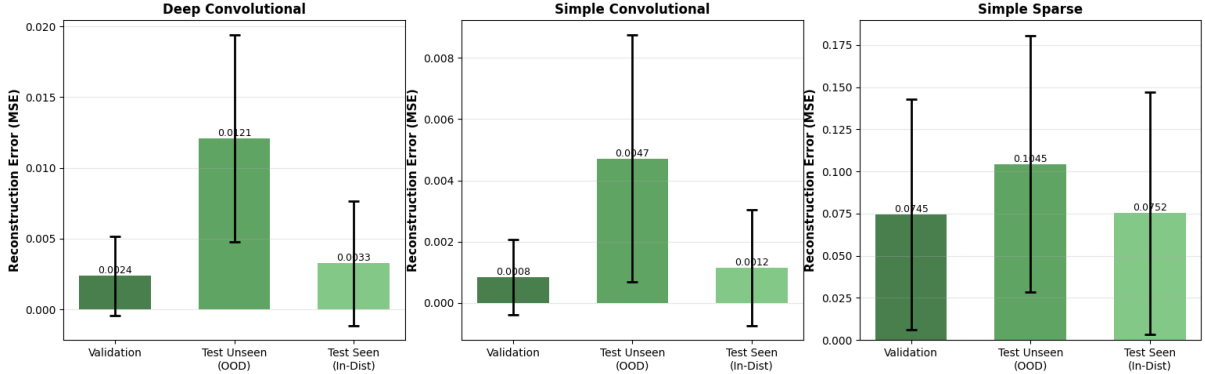


Figure 15: Reconstruction Errors of the Autoencoders

If we analyze the reconstruction error plots shown in Figure 15 for the evaluated data, we observe that in all cases the middle bar is higher. This indicates that all three autoencoders exhibit higher reconstruction errors for unseen data, which is expected, as the models face greater difficulty reconstructing samples that follow a different structure from that of the seen data. However, the third plot corresponding to the simple sparse autoencoder shows that the reconstruction error for seen data—both from the validation set and the test set—remains relatively high, suggesting that this autoencoder also struggles to accurately reconstruct in-distribution samples. In contrast, both convolutional autoencoders present significantly lower reconstruction errors for the validation and test data, indicating a better separation between seen and unseen data.

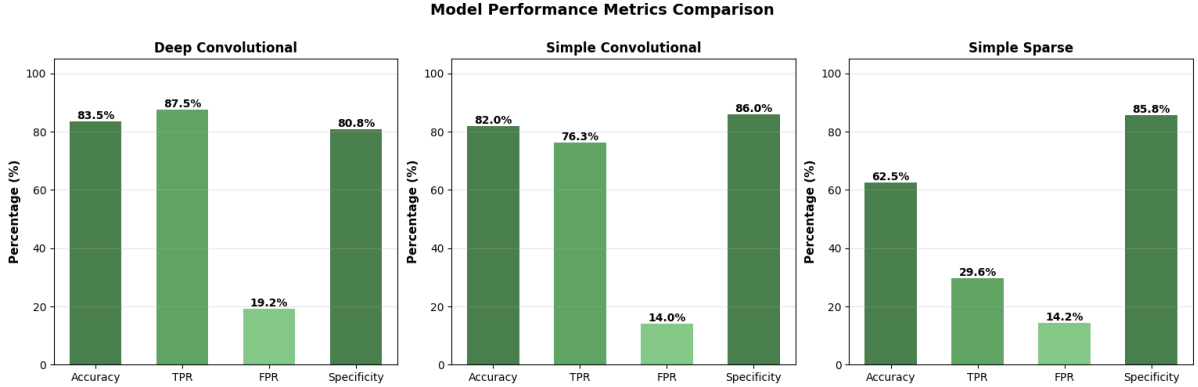


Figure 16: Classification Metrics of the autoencoders on the Test set

If we examine the classification metrics on Figure 16 of the three autoencoder models, we observe that the highest accuracy is achieved by the deep convolutional autoencoder, outperforming the simple convolutional autoencoder by just 1%. This tells us that the deep model classifies both seen and unseen data slightly more effectively in the out-of-distribution detection task.

However, the deep convolutional autoencoder exhibits a higher false positive rate compared to the simple convolutional autoencoder, meaning that it incorrectly classifies a larger number of in-distribution (seen) samples as out-of-distribution. Additionally, its specificity is 5.2% lower, which implies that the model is less effective at correctly identifying in-distribution instances, despite its higher overall accuracy. However, the true positive rate is significantly higher in the deep version.

Finally, the simple sparse autoencoder shows significantly worse performance, achieving a global accuracy of only 62.5% and a true positive rate below 30%, indicating that it struggles

to correctly identify OOD instances. Having stated all these factors, the deep convolutional autoencoder would be the preferred choice for this task.

4.2.3 Qualitative Evaluation

In this section, we provide a qualitative evaluation of the convolutional autoencoders, which were the best-performing models in the OOD detection task. We first show the reconstruction of a test image from a seen class, followed by the reconstruction of an image that resembles this class but belongs to an unseen class, and finally the reconstruction of an image from an unseen class that does not resemble the seen classes, for further analysis.



Figure 17: Example of the reconstruction for the simple convolutional and deep convolutional autoencoders for a test image of a seen class. On the left is the original image, in the middle is the reconstruction from the deep convolutional autoencoder, and on the right is the reconstruction from the simple convolutional autoencoder.



Figure 18: Example of the reconstruction for a test image from an unseen class which resembles to a seen class image using the simple convolutional and deep convolutional autoencoders. On the left is the original image, in the middle is the reconstruction from the deep convolutional autoencoder, and on the right is the reconstruction from the simple convolutional autoencoder.



Figure 19: Example of the reconstruction for a test image from an unseen class which does not resemble to a seen class image using the simple convolutional and deep convolutional autoencoders. On the left is the original image, in the middle is the reconstruction from the deep convolutional autoencoder, and on the right is the reconstruction from the simple convolutional autoencoder.

If we compare Figure 17, we observe that the "50" sign is well reconstructed by both autoencoders, with the deep autoencoder's reconstruction appearing slightly blurrier than that of the simple convolutional autoencoder. In Figure 18, the images are reconstructed reasonably well; however, the deep autoencoder produces a slightly blurrier image and introduces some artifacts that make the number "5" more difficult to interpret. This suggests that the simple convolutional autoencoder may primarily reproduce the original image without fully capturing the underlying patterns, resulting in a more exact—but less abstract—reconstruction.

Finally, Figure 19 illustrates what was suspected in the previous example: the deep convolutional autoencoder produces less precise reconstructions, while the simple convolutional autoencoder appears to reconstruct the images more faithfully. Nevertheless, for the OOD detection task, this qualitative analysis is not critical, although it is interesting. Therefore, we choose the deep convolutional autoencoder for the OOD task.

5 Conclusions

This work has demonstrated that deep learning models can achieve strong performance on both in-distribution classification and out-of-distribution (OOD) detection tasks, while also highlighting important limitations in model reliability. The comparative analysis of CNN, VGG16, and ResNet architectures showed that, although all models reach high accuracy on seen classes, their behavior on unseen data is often characterized by excessive confidence. This overconfidence, observed across all architectures, underscores a critical challenge for deploying such models in real-world scenarios where encountering novel data is unavoidable.

Among the evaluated classifiers, VGG16 exhibited more stable training dynamics and slightly improved uncertainty awareness on OOD samples, as reflected by lower mean confidence and higher entropy. While these improvements are modest, they suggest that architectural simplicity and stability can be advantageous when robustness is a priority. The analysis of misclassification patterns and qualitative examples further revealed that OOD predictions are often driven by shared low-level features such as shape or color, rather than true semantic understanding, which can lead to misleadingly confident predictions.

In parallel, the deep convolutional autoencoder proved effective as a suitable OOD detector, achieving an overall accuracy of 83.5%. This indicates that, in most cases, the autoencoder correctly identifies whether a sample belongs to the in-distribution or not. In the few cases where it fails, a classifier could be applied afterward to assign the proper label.

For the OOD task, it is important to note that we did not perform an exhaustive search for

the optimal reconstruction error threshold on the validation set. Likewise, the reconstruction error itself was not further analyzed as a criterion for improving detection. Future work could explore extending the autoencoder architectures such as VAEs or DAE, trying additional types of training strategies, or experimenting with other hyperparameters to improve performance further along with a deep search of the best threshold.

6 Appendix

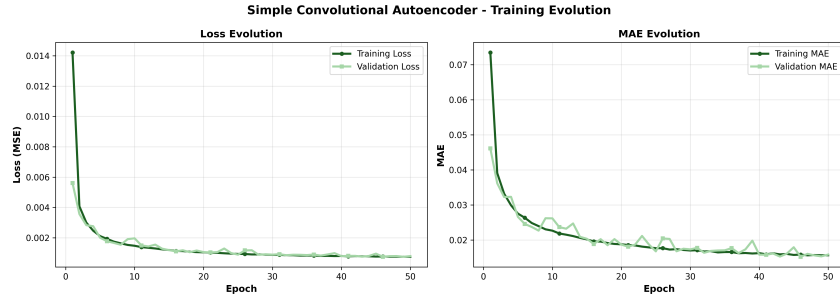


Figure 20: Training Evolution for the Simple Convolutional Autoencoder

Class	Accuracy	Recall	Confidence	Entropy	Samples
0	0.9762	0.9762	0.9857	0.0551	42
1	0.9977	0.9977	0.9994	0.0030	444
2	0.9978	0.9978	0.9992	0.0032	450
3	0.9929	0.9929	0.9941	0.0228	282
4	1.0000	1.0000	1.0000	0.0003	396
5	0.9973	0.9973	0.9978	0.0067	372
6	1.0000	1.0000	1.0000	0.0000	84
7	1.0000	1.0000	1.0000	0.0001	288
8	1.0000	1.0000	1.0000	0.0000	282
9	1.0000	1.0000	1.0000	0.0001	294
10	1.0000	1.0000	1.0000	0.0000	402
11	1.0000	1.0000	1.0000	0.0000	264
12	1.0000	1.0000	1.0000	0.0000	420
13	1.0000	1.0000	1.0000	0.0000	432
14	1.0000	1.0000	1.0000	0.0002	156
15	1.0000	1.0000	1.0000	0.0000	126
16	1.0000	1.0000	1.0000	0.0000	84
17	1.0000	1.0000	1.0000	0.0000	222
18	1.0000	1.0000	1.0000	0.0000	240
19	1.0000	1.0000	1.0000	0.0000	42
20	1.0000	1.0000	1.0000	0.0001	72
21	1.0000	1.0000	0.9874	0.0471	66
22	1.0000	1.0000	1.0000	0.0000	78
23	1.0000	1.0000	1.0000	0.0000	102
24	1.0000	1.0000	0.9958	0.0216	54
25	1.0000	1.0000	1.0000	0.0000	300
26	0.9917	0.9917	0.9971	0.0069	120
27	1.0000	1.0000	1.0000	0.0000	48
28	1.0000	1.0000	1.0000	0.0001	108
29	1.0000	1.0000	1.0000	0.0002	54
30	0.9889	0.9889	0.9978	0.0056	90
31	1.0000	1.0000	1.0000	0.0002	156
32	1.0000	1.0000	1.0000	0.0000	48
33	1.0000	1.0000	1.0000	0.0000	138
34	1.0000	1.0000	1.0000	0.0000	84
35	1.0000	1.0000	1.0000	0.0000	240

Class	Accuracy	Recall	Confidence	Entropy	Samples
36	0.9872	0.9872	0.9986	0.0045	78
37	1.0000	1.0000	1.0000	0.0000	42
38	1.0000	1.0000	1.0000	0.0002	414
39	1.0000	1.0000	1.0000	0.0000	60
40	1.0000	1.0000	1.0000	0.0000	72
41	1.0000	1.0000	0.9999	0.0004	48
42	1.0000	1.0000	1.0000	0.0000	48

Table 10: VGG11 Per-Class Metrics on Seen Classes

Class	Accuracy	Recall	Confidence	Entropy	Samples
0	0.8810	0.8810	0.9601	0.1103	42
1	0.9977	0.9977	0.9991	0.0022	444
2	0.9978	0.9978	0.9981	0.0050	450
3	1.0000	1.0000	0.9960	0.0135	282
4	0.9722	0.9722	0.9859	0.0348	396
5	0.9919	0.9919	0.9952	0.0170	372
6	1.0000	1.0000	0.9999	0.0005	84
7	1.0000	1.0000	0.9999	0.0005	288
8	0.9752	0.9752	0.9762	0.0622	282
9	1.0000	1.0000	1.0000	0.0001	294
10	1.0000	1.0000	0.9995	0.0020	402
11	1.0000	1.0000	0.9998	0.0011	264
12	1.0000	1.0000	0.9997	0.0011	420
13	1.0000	1.0000	0.9989	0.0025	432
14	1.0000	1.0000	0.9969	0.0063	156
15	1.0000	1.0000	0.9997	0.0022	126
16	1.0000	1.0000	1.0000	0.0002	84
17	1.0000	1.0000	0.9999	0.0007	222
18	1.0000	1.0000	0.9997	0.0020	240
19	1.0000	1.0000	0.9998	0.0015	42
20	1.0000	1.0000	0.9902	0.0320	72
21	0.9697	0.9697	0.9806	0.0615	66
22	1.0000	1.0000	0.9967	0.0129	78
23	1.0000	1.0000	0.9998	0.0014	102
24	0.9815	0.9815	0.9875	0.0408	54
25	0.9867	0.9867	0.9971	0.0114	300
26	1.0000	1.0000	0.9926	0.0214	120
27	1.0000	1.0000	0.9912	0.0363	48
28	1.0000	1.0000	0.9983	0.0078	108
29	1.0000	1.0000	0.9999	0.0007	54
30	0.9889	0.9889	0.9923	0.0287	90
31	1.0000	1.0000	0.9997	0.0016	156
32	1.0000	1.0000	1.0000	0.0000	48
33	1.0000	1.0000	0.9998	0.0016	138
34	1.0000	1.0000	0.9994	0.0038	84
35	1.0000	1.0000	0.9998	0.0013	240
36	1.0000	1.0000	1.0000	0.0004	78
37	1.0000	1.0000	0.9988	0.0067	42
38	1.0000	1.0000	0.9987	0.0032	414

Class	Accuracy	Recall	Confidence	Entropy	Samples
39	1.0000	1.0000	0.9998	0.0013	60
40	1.0000	1.0000	0.9995	0.0030	72
41	1.0000	1.0000	0.9860	0.0294	48
42	1.0000	1.0000	0.9998	0.0013	48

Table 11: ResNet Per-Class Metrics on Seen Classes

Class	Accuracy	Recall	Confidence	Entropy	Samples
0	0.9524	0.9524	0.8433	0.4764	42
1	0.9820	0.9820	0.9510	0.1643	444
2	0.9467	0.9467	0.8939	0.3339	450
3	0.9468	0.9468	0.8851	0.3909	282
4	0.9747	0.9747	0.9180	0.2877	396
5	0.9435	0.9435	0.8651	0.4492	372
6	1.0000	1.0000	0.9573	0.1762	84
7	0.9375	0.9375	0.8785	0.3698	288
8	0.8972	0.8972	0.8380	0.4860	282
9	0.9626	0.9626	0.9326	0.2272	294
10	0.9826	0.9826	0.9613	0.1582	402
11	0.9962	0.9962	0.9544	0.1867	264
12	1.0000	1.0000	0.9932	0.0353	420
13	0.9977	0.9977	0.9836	0.0741	432
14	0.9808	0.9808	0.9798	0.0602	156
15	1.0000	1.0000	0.9643	0.1478	126
16	0.9881	0.9881	0.9415	0.2115	84
17	0.9955	0.9955	0.9939	0.0274	222
18	0.9958	0.9958	0.9396	0.2427	240
19	0.9762	0.9762	0.7404	0.8514	42
20	0.9583	0.9583	0.7789	0.7716	72
21	0.9697	0.9697	0.8747	0.4653	66
22	0.9872	0.9872	0.9441	0.2253	78
23	0.9902	0.9902	0.8896	0.4267	102
24	0.9074	0.9074	0.8455	0.5321	54
25	0.9700	0.9700	0.9408	0.2232	300
26	0.9333	0.9333	0.8717	0.4528	120
27	0.9375	0.9375	0.8440	0.5076	48
28	0.9907	0.9907	0.9168	0.3009	108
29	0.9074	0.9074	0.8828	0.3950	54
30	0.9000	0.9000	0.8310	0.5953	90
31	0.9872	0.9872	0.8984	0.4037	156
32	1.0000	1.0000	0.9429	0.2502	48
33	0.9783	0.9783	0.9208	0.2782	138
34	0.9524	0.9524	0.8844	0.3756	84
35	0.9958	0.9958	0.9577	0.1611	240
36	0.9231	0.9231	0.8620	0.4606	78
37	0.9524	0.9524	0.8762	0.4377	42
38	1.0000	1.0000	0.9891	0.0528	414
39	1.0000	1.0000	0.9390	0.1876	60
40	0.9722	0.9722	0.9449	0.2069	72
41	0.9792	0.9792	0.8183	0.5763	48
42	0.9792	0.9792	0.8087	0.6168	48

Table 12: CNN Per-Class Metrics on Seen Classes